Master Method (Master Theorem)

Classify divide-and-conquer recurrences quickly — after checking that the required form and conditions hold

IDENTIFY → COMPARE → SOLVE

$T(n)=aT(n/b)+f(n) \rightarrow \text{benchmark} \rightarrow \text{case} \rightarrow \Theta(\cdot)$
Lecture 10 showed the pattern with trees. Lecture 11 turns that pattern into a fast classification method.

Learning outcomes

Lecture 11 — Master Method for Recurrences

Identify a, b, and f(n) in $T(n)=aT(n/b)+f(n)$.
Explain the three principal cases and their conditions.
Match a recurrence to the correct case.
Calculate the resulting asymptotic complexity.
Recognize recurrences unsuitable for direct application.
Compare Master Method with substitution and recurrence trees.

Bridge from Lecture 10: what the trees already showed

Now we compress the recurrence-tree patterns into one reusable rule.

Does NOT fit

$T(n)=T(n-1)+1$

Shrink is $n-1$, not n/b.
Use expansion/substitution.

Fits Master form

$T(n)=T(n/2)+1$

$a=1$, $b=2$, $f(n)=1$
Result: $\Theta(\log n)$

Fits Master form

$T(n)=2T(n/2)+n$

$a=2$, $b=2$, $f(n)=n$
Result: $\Theta(n \log n)$

Master Method: a shortcut for a common recurrence form

It works when the recurrence matches the divide-and-conquer pattern.

MASTER

$$T(n) = aT(n/b) + f(n)$$

a = number of subproblems • n/b = size of each subproblem • f(n) = current-call work

The theorem compares f(n) with the recursive branching term n^(log_b a).

If the recurrence does not fit this form, use another method.

What does a mean?

a tells us how many recursive subproblems each call creates.

$T(n)=2T(n/2)+n$

a = 2

Two recursive subproblems are created.

$T(n)=4T(n/2)+n$

a = 4

Four recursive subproblems are created.

What does b mean?

b tells us how much smaller each recursive subproblem becomes.

$T(n)=2T(n/2)+n$

b = 2

Each subproblem has size n/2.

$T(n)=3T(n/3)+n$

b = 3

Each subproblem has size n/3.

What does f(n) mean?

f(n) is the work done by the current call outside the recursive calls.

+1

Constant current-call work

+n

Linear current-call work

+n^2

Quadratic current-call work

How a and b shape the recursion tree

a controls how many branches appear; b controls how quickly each branch becomes smaller.

a = number of recursive calls

$$T(n)=3T(n/2)+n$$

Each call creates 3 children.
After i levels: 3^i nodes.

Larger a → more branching → potentially much more total work.

b = shrink factor

$$T(n)=3T(n/4)+n$$

Each child has size n/4.
After i levels: $n/4^i$.

Larger b → problem shrinks faster → fewer recursion levels.

Do not confuse them

$$T(n)=4T(n/2)+n$$

a = 4

because FOUR recursive calls are created

b = 2

because each call gets n/2 input

Think: a = branches, b = shrinkage.

Extract a, b, and f(n) before doing any theorem work

Read the recurrence structurally first. The case comes later.

Example 1

$$T(n)=2T(n/2)+n$$

$a=2$
$b=2$
$f(n)=n$

Example 2

$$T(n)=4T(n/2)+n$$

$a=4$
$b=2$
$f(n)=n$

Example 3

$$T(n)=3T(n/3)+n^2$$

$a=3$
$b=3$
$f(n)=n^2$

Never choose Case 1, 2, or 3 until a, b, and f(n) are correct.

Where does n^(log_b a) actually come from?

It is not a magic expression — it comes directly from the branching and the tree depth.

1 — tree depth

At depth h:

$$n / b^h = 1$$

So:

$$h = \log_b n$$

2 — nodes at that depth

Each level multiplies nodes by a.

$$\text{leaves} = a^h$$

Substitute h = log_b n:

$$a^{(\log_b n)}$$

3 — rewrite the expression

Using the logarithm identity:

$$a^{(\log_b n)} = n^{(\log_b a)}$$

So n^(log_b a) represents the growth created by the recursive branching structure.

Master Method compares f(n) against this recursive-growth baseline.

The comparison benchmark

Master Method compares f(n) with n^(log_b a).

n^(log_b a)

This represents the growth caused by the recursive branching structure.

f(n) smaller

Recursive branching dominates.

same order

Both contribute at the same scale.

f(n) larger

Current-call work dominates.

Case 1 — recursive branching dominates

Condition: f(n)=O(n^(log_b a − ε)) for some ε>0.

$$T(n)=4T(n/2)+n$$

$a=4$, $b=2$, $f(n)=n$

Compute the benchmark

$n^{(\log_2 4)} = n^2$

Compare

$f(n)=n$ is smaller than n^2

$$T(n)=\Theta(n^2)$$

The recursive branching term wins.

Case 2 — same-order contributions

Simple Case 2: f(n)=Θ(n^(log_b a)).

T(n)=2T(n/2)+n

Merge Sort: a=2, b=2, f(n)=n

Benchmark

$n^{(\log_2 2)} = n$

Compare

f(n)=n matches the benchmark

T(n)=Θ(n log n)

An extra log n factor appears.

Case 3 — current-call work dominates

Conditions: f(n)=Ω(n^(log_b a + ε)) for some ε>0, plus regularity.

$$T(n)=2T(n/2)+n^2$$

$a=2$, $b=2$, $f(n)=n^2$

Benchmark

$n^{(\log_2 2)} = n$

Compare

$f(n)=n^2$ is larger than n

$$T(n)=\Theta(n^2)$$

The non-recursive work wins when the regularity condition also holds.

Case 3 regularity condition

Why do we need one extra check?

$$a\,f(n/b) \leq c\,f(n), \quad \text{for some } c < 1$$

The recursive children must not collectively reproduce almost all of the parent's f(n) work.

For $2T(n/2)+n^2$

$2 \times (n/2)^2 = n^2/2$.
Choose c = 1/2, which is less than 1. The condition holds.

A simple Master Method decision flow

Compare f(n) with n^(log_b a).

Step 1

Compute n^(log_b a)



Case 1

f(n) smaller
→ Θ(n^(log_b a))

Case 2

same order
→ add a log factor

Case 3

f(n) larger
→ Θ(f(n)), if regularity holds

Always identify a, b, and f(n) before deciding a case.

Before analyzing Merge Sort — understand the algorithm first

See what one Merge Sort call does before analyzing its recurrence.

Problem

Sort n values into nondecreasing order.

Input

[8, 3, 6, 2, 7, 1, 5, 4]

Output

[1, 2, 3, 4, 5, 6, 7, 8]

Three ideas

1 DIVIDE	Split the array into two halves.
2 CONQUER	Recursively sort each half.
3 COMBINE	Merge the two sorted halves.

First understand the split and merge. Then write T(n).

Merge Sort Step 1 — repeatedly divide into halves

Keep splitting until every subproblem contains one element.

n = 8

[8, 3, 6, 2, 7, 1, 5, 4]

↓ split each current subproblem

n = 4

[8, 3, 6, 2] **[7, 1, 5, 4]**

↓ split each current subproblem

n = 2

[8, 3] **[6, 2]** **[7, 1]** **[5, 4]**

↓ split each current subproblem

n = 1

[8] [3] [6] [2] [7] [1] [5] [4]

Base case: a one-element list is already sorted.

Merge Sort Step 2 — merge the sorted pieces back upward

At every merge, repeatedly take the smaller front element.

Singles [8] [3] [6] [2] [7] [1] [5] [4]

↓ merge neighboring sorted lists

Merge to size 2 [3,8] [2,6] [1,7] [4,5]

↓ merge neighboring sorted lists

Merge to size 4 [2,3,6,8] [1,4,5,7]

↓ merge neighboring sorted lists

Final merge [1,2,3,4,5,6,7,8]

The algorithm splits downward, then does its important non-recursive work while merging upward.

What actually happens inside one merge?

A concrete merge explains why combining two halves costs linear work.

Left sorted list **[2, 6]**

Right sorted list **[3, 8]**

1	**Compare 2 and 3**	2 is smaller → output [2]
2	**Compare 6 and 3**	3 is smaller → output [2,3]
3	**Compare 6 and 8**	6 is smaller → output [2,3,6]
4	**Left list empty**	append remainder 8 → [2,3,6,8]

Across a merge of m total elements, each element is processed only a constant number of times → Θ(m).

Now derive Merge Sort's recurrence from the algorithm

Every term comes directly from Divide, Conquer, or Combine.

DIVIDE

Split at the middle

Θ(1) in our simplified model

CONQUER

Sort left half + sort right half

2T(n/2)

COMBINE

Merge two sorted halves

Θ(n)

$$T(n) = 2T(n/2) + \Theta(n)$$

Now it is meaningful to compare expansion, recurrence trees, and the Master Method.

One recurrence, three solving techniques: Merge Sort

Use exactly the same recurrence so we can compare the amount of work each analysis method requires.

$$T(n) = 2T(n/2) + n, \quad T(1)=1$$

Expansion / substitution

Unroll the recurrence until a pattern appears.

Recurrence tree

See cost level by level and add the levels.

Master Method

Identify a,b,f(n), compare, and classify quickly.

All three must end at the same answer: Θ(n log n).

Merge Sort by expansion: expose the repeated algebra

This method is excellent for understanding how the recurrence unfolds.

$$T(n) = 2T(n/2) + n$$

Expand one step at a time

$T(n) = 2[T(n/2)] + n$

$= 2[2T(n/4) + n/2] + n$

$= 4T(n/4) + 2n$

$= 4[2T(n/8) + n/4] + 2n$

$= 8T(n/8) + 3n$

…

After k expansions: $T(n) = 2^kT(n/2^k) + kn$

Pattern found — but we still need the stopping value k.

Merge Sort by expansion: stop at the base case

Now finish the algebra carefully instead of jumping directly to the answer.

$$T(n) = 2^kT(n/2^k) + kn$$

Find k

Stop when $n/2^k = 1$

$n = 2^k$

$k = \log_2 n$

Substitute k

$T(n) = 2^{(\log_2 n)}T(1) + n \log_2 n$

$= n \cdot 1 + n \log_2 n$

$= n + n \log_2 n$

Therefore: $\Theta(n \log n)$

Correct and informative — but several algebraic steps are required.

Merge Sort by recurrence tree: make the work visible

The tree gives a visual explanation of why n log n appears.

Level 0	**1 node × n**	**level cost = n**
Level 1	**2 nodes × n/2**	**level cost = n**
Level 2	**4 nodes × n/4**	**level cost = n**
Level i	**2^i nodes × $n/2^i$**	**level cost = n**

Finish the tree

Depth = $\log_2 n$

Each internal level costs n

Internal work = $n \log_2 n$

Leaves = $n \times \Theta(1) = \Theta(n)$

Total = $\Theta(n \log n)$

Excellent explanation — but drawing / summing a tree every time takes work.

Merge Sort by Master Method: same answer in four steps

Once the recurrence is understood, the theorem can classify it immediately.

1 — Identify

$a = 2$
$b = 2$
$f(n) = n$

2 — Benchmark

$n^{(\log_b a)}$
$= n^{(\log_2 2)}$
$= n$

3 — Compare

$f(n)=n$ matches benchmark n
→ Case 2

4 — Result

$T(n)=\Theta(n \log n)$

No expansion. No full tree. The theorem summarizes a pattern we already understand.

Why use Master Method sometimes — and not always?

The fastest analysis technique depends on the recurrence, just like algorithm choice depends on the workload.

Expansion / substitution

Best when:

- recurrence is simple to unroll
- Master form does not fit
- you want an exact / near-exact pattern

Strength:

shows the algebra directly

Recurrence tree

Best when:

- you want to see level costs
- branching behavior matters
- students are building intuition

Strength:

explains where the total comes from

Master Method

Best when:

- recurrence fits $aT(n/b)+f(n)$
- you need a fast Θ classification
- case conditions are easy to verify

Strength:

very short once the model is correct

Understand with expansion/tree first; use Master Method as the shortcut when it fits.

When the Master Method does not apply directly

The recurrence must match aT(n/b)+f(n).

$T(n)=T(n-1)+1$

Subproblem size is $n-1$, not n/b.

Use substitution or another method.

$T(n)=T(n/3)+T(2n/3)+n$

Subproblems have different sizes.

The standard Master form does not fit.

Before applying the method

First verify equal-size recursive subproblems and the exact form $aT(n/b)+f(n)$.

Practice: choose the case and final Θ class

Work from a, b, f(n), and the benchmark.

A

$T(n)=8T(n/2)+n^2$

Case: _____
Θ: _____

B

$T(n)=3T(n/3)+n$

Case: _____
Θ: _____

C

$T(n)=2T(n/2)+n^3$

Case: _____
Θ: _____

Practice answers

Check the benchmark before reading the case number.

A

Benchmark: n^3
$f(n)=n^2$ is smaller

Case 1
$\Theta(n^3)$

B

Benchmark: n
$f(n)=n$ matches

Case 2
$\Theta(n \log n)$

C

Benchmark: n
$f(n)=n^3$ is larger

Case 3
$\Theta(n^3)$

Which recurrence-solving method should you use?

All three methods analyze the same recurrence from different angles.

Method	Best use	Main advantage
Substitution	You can guess a bound	Can prove the bound
Recurrence Tree	You want to see level costs	Very visual
Master Method	Recurrence fits aT(n/b)+f(n)	Fast classification

A fast method is useful only when its assumptions fit the recurrence.

Common mistakes to avoid

Slow down at the comparison step.

Using a before identifying how many recursive calls are actually made.
Confusing a and b — a is branch count; b is the shrink factor.
Computing log_b a incorrectly.
Choosing a case before comparing f(n) with n^(log_b a).
Applying the method to T(n−1) or unequal-size subproblems.
Ignoring the Case 3 regularity condition.

Application case study: Strassen’s Matrix Multiplication

First do ordinary 2×2 multiplication completely. Then do the same product with Strassen and compare.

A = [[1, 2], [3, 4]]

B = [[5, 6], [7, 8]]

Goal

Compute C = AB using both methods and verify that both produce exactly the same matrix.

Ordinary method → 8 multiplications. Strassen → 7 multiplications.

Conventional matrix multiplication — formula before the numbers

Each output entry is the dot product of one row of A with one column of B.

General n×n formula

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

row i × column j

For 2×2 matrices

$A = [[a, b], [c, d]]$ $B = [[e, f], [g, h]]$

$$C = AB = [[ae + bg,\ af + bh],\ [ce + dg,\ cf + dh]]$$

Four output cells × two scalar products per cell = 8 scalar multiplications.

Ordinary 2×2 multiplication: where every output entry comes from

Each output cell is a row-by-column dot product.

A = [[1,2],[3,4]]

B = [[5,6],[7,8]]

Top row of C

c_{11} = (1×5) + (2×7)

= 5 + 14

= 19

c_{12} = (1×6) + (2×8)

= 6 + 16

= 22

Bottom row of C

c_{21} = (3×5) + (4×7)

= 15 + 28

= 43

c_{22} = (3×6) + (4×8)

= 18 + 32

= 50

C = [[19,22],[43,50]]

Ordinary 2×2 multiplication: operation count

This tiny example reveals the recursive pattern used for larger matrices.

For the 2×2 scalar example

c_{11} uses 2 multiplications + 1 addition

c_{12} uses 2 multiplications + 1 addition

c_{21} uses 2 multiplications + 1 addition

c_{22} uses 2 multiplications + 1 addition

TOTAL: 8 multiplications + 4 additions

For recursive block matrices

Split A and B into four n/2 × n/2 blocks.

Each C block needs 2 recursive block multiplications.

4 output blocks × 2 = 8 recursive multiplications.

Block additions contribute $\Theta(n^2)$ extra work.

$$T(n)=8T(n/2)+\Theta(n^2)$$

Master Method: $a=8$, $b=2$, $f(n)=n^2 \rightarrow \Theta(n^3)$.

Strassen starts from the same four matrix blocks

The result C=AB is unchanged. Only the way we compute the block products changes.

A

$$\begin{matrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{matrix}$$

B

$$\begin{matrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{matrix}$$

For our numerical example

$A_{11}=1$, $A_{12}=2$

$A_{21}=3$, $A_{22}=4$

$B_{11}=5$, $B_{12}=6$

$B_{21}=7$, $B_{22}=8$

Strassen forms 7 carefully chosen products M_1 … M_7.

The extra additions/subtractions let us avoid the 8th multiplication.

Strassen’s algorithm — the seven multiplication formulas

These formulas replace the eight ordinary block multiplications with seven carefully chosen products.

Products M_1–M_4

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

Products M_5–M_7

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

The additions/subtractions cost extra work, but one recursive multiplication disappears.

Strassen’s algorithm — recombine $M_1 \ldots M_7$ into C

After the seven products are available, construct the four output blocks with these formulas.

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

These formulas are the algorithm. The following slides substitute actual numbers into every one of them.

Strassen numerical example: compute M_1, M_2, M_3

Work slowly — every symbol now has a concrete number.

M_1

$$(A_{11}+A_{22})(B_{11}+B_{22})$$

$$(1+4)(5+8)$$
$$= 5\times13$$
$$= 65$$

M_2

$$(A_{21}+A_{22})B_{11}$$

$$(3+4)\times5$$
$$= 7\times5$$
$$= 35$$

M_3

$$A_{11}(B_{12}-B_{22})$$

$$1\times(6-8)$$
$$= 1\times(-2)$$
$$= -2$$

So far: $M_1=65$, $M_2=35$, $M_3=-2$.

Strassen numerical example: compute M_4, M_5, M_6, M_7

These four products complete the seven-multiplication set.

M_4

$$A_{22}(B_{21}-B_{11})$$

$4\times(7-5)$
$= 4\times2$
$= 8$

M_5

$$(A_{11}+A_{12})B_{22}$$

$(1+2)\times8$
$= 3\times8$
$= 24$

M_6

$$(A_{21}-A_{11})(B_{11}+B_{12})$$

$(3-1)(5+6)$
$= 2\times11$
$= 22$

M_7

$$(A_{12}-A_{22})(B_{21}+B_{22})$$

$(2-4)(7+8)$
$= -2\times15$
$= -30$

$M_4=8$, $M_5=24$, $M_6=22$, $M_7=-30$.

Strassen recombination: calculate C_{11} and C_{12}

Now combine the seven intermediate products into the first row of C.

C_{11}

$$C_{11} = M_1 + M_4 - M_5 + M_7$$
$$= 65 + 8 - 24 + (-30)$$
$$= 73 - 24 - 30$$
$$= 49 - 30$$
$$= \mathbf{19}$$

C_{12}

$$C_{12} = M_3 + M_5$$
$$= -2 + 24$$
$$= \mathbf{22}$$

First row obtained: [19, 22]

Strassen recombination: calculate C_{21} and C_{22}

Complete the second row and verify the full product.

C_{21}

$$C_{21} = M_2 + M_4$$
$$= 35 + 8$$
$$= \mathbf{43}$$

C_{22}

$$C_{22} = M_1 - M_2 + M_3 + M_6$$
$$= 65 - 35 - 2 + 22$$
$$= 30 - 2 + 22$$
$$= 28 + 22$$
$$= \mathbf{50}$$

C = [[19,22],[43,50]] ✓ same as ordinary multiplication

Ordinary vs Strassen on the 2×2 example

Strassen saves one multiplication but pays for many more additions/subtractions.

Ordinary

8 scalar multiplications

4 scalar additions

simple and low overhead

Strassen

7 scalar multiplications

18 additions/subtractions*

less multiplication, more overhead

*Using the straightforward $M_1 \ldots M_7$ formulation shown in these slides.

For tiny matrices ordinary multiplication may be faster; recursion changes the story for large n.

From the 2×2 trick to an n×n recurrence

When blocks are matrices, each M_i becomes one recursive multiplication of size n/2.

What repeats recursively

7 products M_1 … M_7

→ 7 recursive multiplications

→ each on (n/2)×(n/2) matrices

So recursive part = 7T(n/2)

What happens outside recursion

A constant number of matrix additions/subtractions

Each touches $\Theta((n/2)^2)$ entries

Constant × $\Theta(n^2/4) = \Theta(n^2)$

So $f(n)=\Theta(n^2)$

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Use a and b on Strassen — carefully

Identify a, b, and f(n) directly from Strassen's recurrence.

a = 7

Why?

Each Strassen call creates SEVEN recursive matrix multiplications.

b = 2

Why?

Each recursive matrix has side length n/2.

$f(n) = \Theta(n^2)$

Why?

Matrix additions and subtractions process $\Theta(n^2)$ entries overall.

a counts recursive calls; b describes input shrinkage.

Master Method calculation for Strassen: every step

Do the comparison explicitly before naming the case.

$$T(n)=7T(n/2)+\Theta(n^2)$$

Step 1 — benchmark

$n^{\log_b a}$

$= n^{\log_2 7}$

$\approx n^{2.807}$

Step 2 — compare

$f(n)=n^2$

n^2 grows polynomially slower than $n^{2.807}$

→ Case 1

Step 3 — result

$T(n)=\Theta(n^{\log_2 7})$

$\approx \Theta(n^{2.807})$

2.807 < 3, so Strassen grows asymptotically more slowly than ordinary multiplication.

Why 8 recursive multiplications vs 7 becomes a big difference

The saving repeats at every level of recursion.

Ordinary recursive multiplication

$$T(n)=8T(n/2)+\Theta(n^2)$$

At depth k: about 8^k recursive products

For $n=2^k$, $8^k = n^3$

$n=1024=2^{10} \rightarrow 8^{10} = 1{,}073{,}741{,}824$

Strassen

$$T(n)=7T(n/2)+\Theta(n^2)$$

At depth k: about 7^k recursive products

For $n=2^k$, $7^k = n^{\wedge}(\log_2 7)$

$n=1024=2^{10} \rightarrow 7^{10} = 282{,}475{,}249$

At this depth, Strassen has about 3.8× fewer leaf multiplications.

Practical lesson: asymptotically better does not mean always faster

Asymptotic improvement must still be balanced against practical overhead.

Why Strassen can win

- **exponent falls from 3 to ≈2.807**
- multiplication saving compounds recursively
- advantage grows with matrix size
- demonstrates how algebra can change algorithmic complexity

Why ordinary multiplication may still win

- **Strassen performs many more additions/subtractions**
- extra temporary storage / recursive overhead
- padding and implementation complexity
- small matrices may favor the simpler method

For sufficiently large matrices, the lower asymptotic exponent can outweigh the extra overhead.

Final summary

Lecture 11 checklist: verify the form, compare growth, then classify.

Before Master Method

Understand where the recurrence comes from.
Use substitution/tree when the Master form does not fit.
Do not force a theorem onto the wrong recurrence.

Master Method

Identify a, b, f(n).
Compute n^(log_b a).
Check the case conditions.
State the tight Θ result.

Fast classification comes after correct modeling.

Next lecture: Brute Force Pattern Matching.